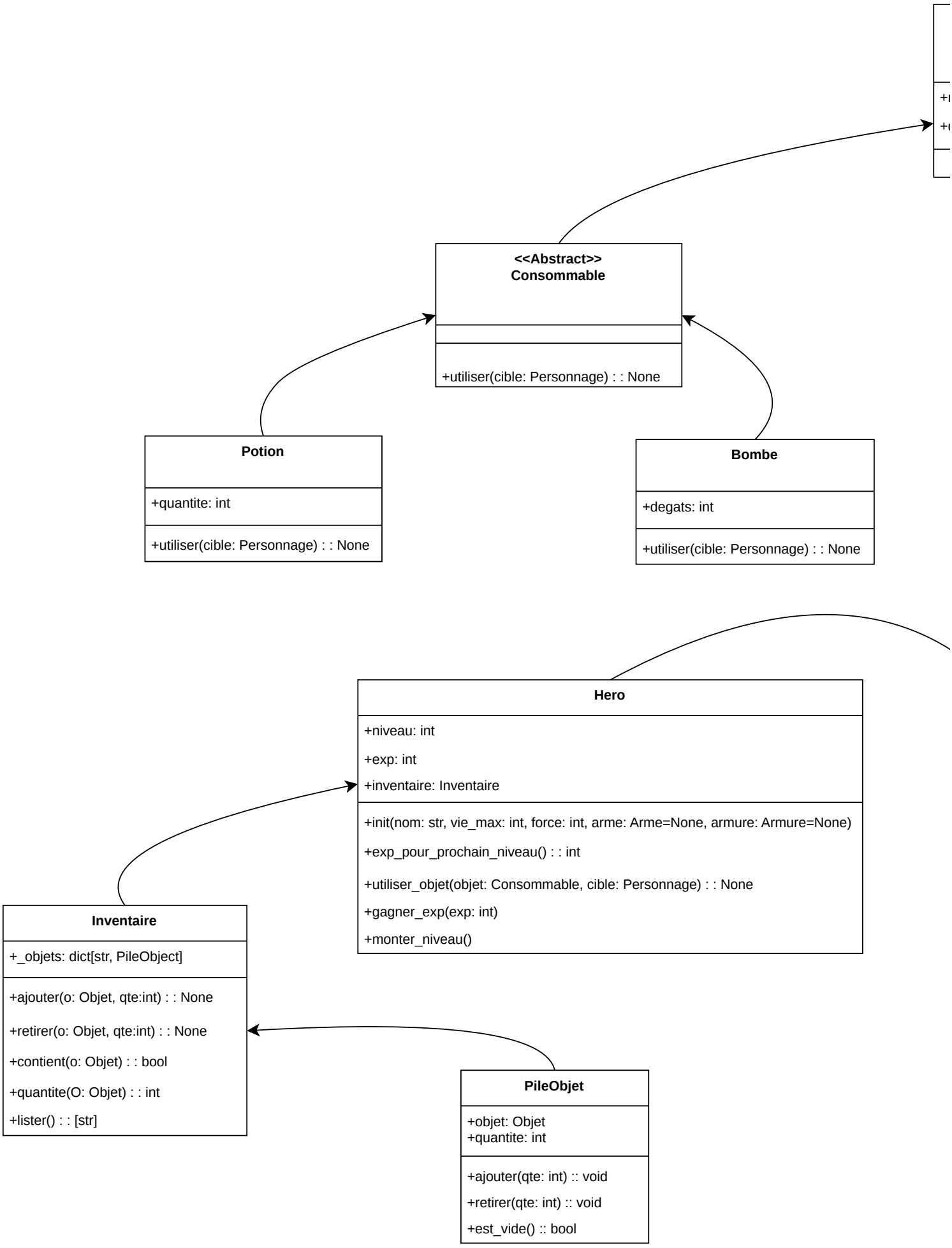
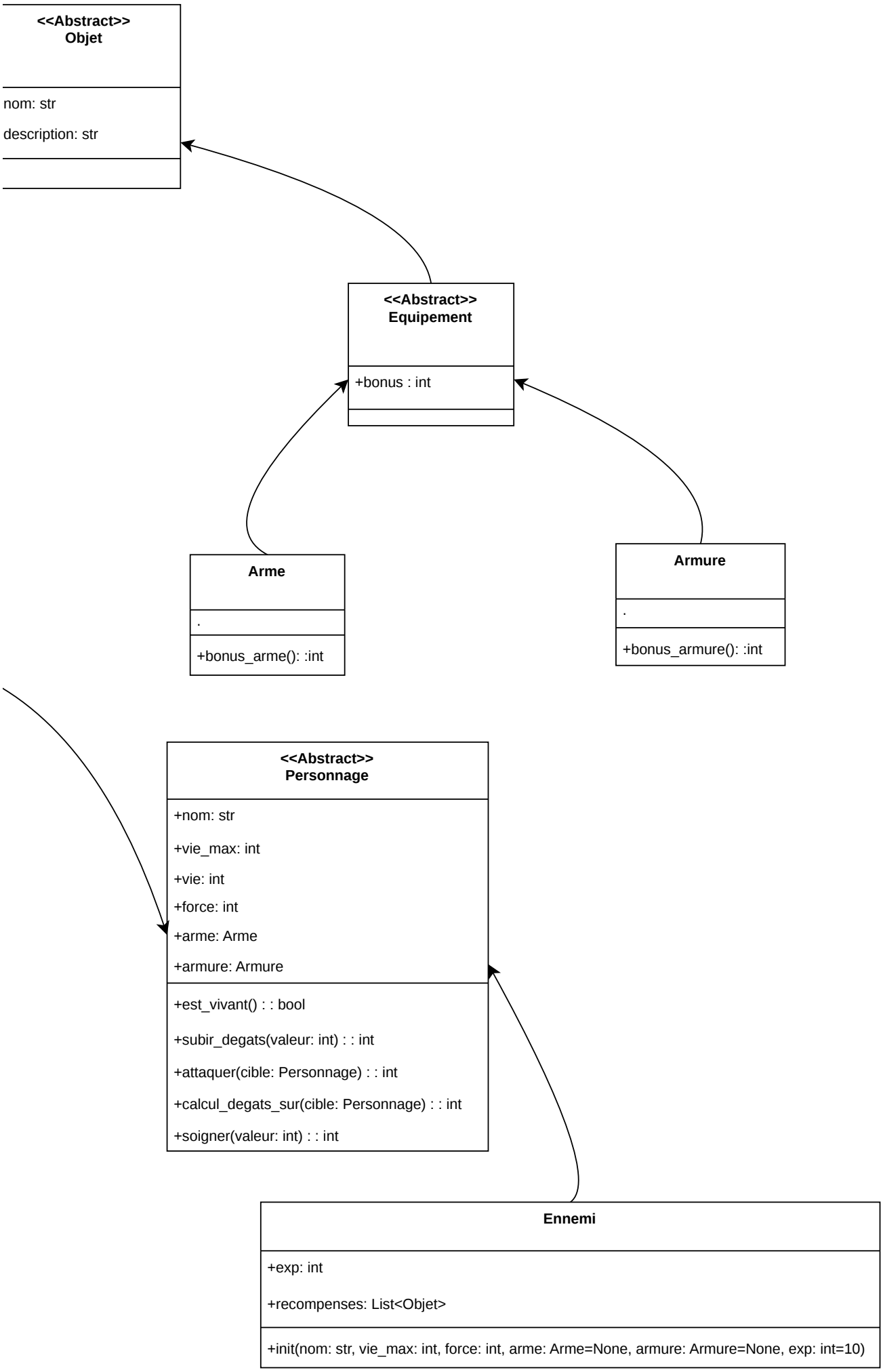

<<Abstract>>
Consommable
+utiliser(cible: Personnage) : : None
Potion
+quantite: int
+utiliser(cible: Personnage) : : None
Bombe
+degats: int
+utiliser(cible: Personnage) : : None
Hero
+niveau: int
+exp: int
+inventaire: Inventaire
+init(nom: str, vie_max: int, force: int, arme: Arme=None, armure: Armure=None)
+exp_pour_prochain_niveau() : : int
+utiliser_objet(objet: Consommable, cible: Personnage) : : None
+gagner_exp(exp: int)
+monter_niveau()
Inventaire
+_objets: dict[str, PileObject]
+ajouter(o: Objet, qte:int) : : None
+retirer(o: Objet, qte:int) : : None
+contient(o: Objet) : : bool
+quantite(O: Objet) : : int
+lister() : : [str]
PileObjet
+objet: Objet
+quantite: int
+ajouter(qte: int) :: void
+retirer(qte: int) :: void
+est_vide() :: bool


<<Abstract>>
Objet
nom: str
description: str
<<Abstract>>
Equipement
+bonus : int
Arme
.
+bonus_arme(): :int
Armure
.
+bonus_armure(): :int
<<Abstract>>
Personnage
+nom: str
+vie_max: int
+vie: int
+force: int
+arme: Arme
+armure: Armure
+est_vivant() : : bool
+subir_degats(valeur: int) : : int
+attaquer(cible: Personnage) : : int
+calcul_degats_sur(cible: Personnage) : : int
+soigner(valeur: int) : : int
Ennemi
+exp: int
+recompenses: List<Objet>
+init(nom: str, vie_max: int, force: int, arme: Arme=None, armure: Armure=None, exp: int=10)